

Introduction to Python

Module 7 — pandas

DataFrames, filtering, grouping, and plotting

What Is Pandas?

pandas is *the* library for working with data tables in Python — think of it as a programmable spreadsheet. Its core object is the **DataFrame**: rows and columns, like an Excel sheet.

```
import pandas as pd

df = pd.DataFrame({
    "city": ["Wageningen", "Utrecht"],
    "population": [40000, 360000]
})
```

`import pandas as pd` is the universal convention. A **DataFrame** has **named columns** and **indexed rows** — and hundreds of methods for slicing, filtering, and summarizing.

Loading Real Data

Most of the time you don't type data by hand — you **load a file**. pandas reads CSVs (and Excel, JSON, and more) in one line.

```
df = pd.read_csv("data.csv")

df.head()      # first 5 rows
df.shape       # (rows, columns)
df.columns     # the column names
df.info()      # types & missing values
df.describe()  # summary statistics
```

head(), info(), and describe() are the first three things you run on any new dataset — always look before you leap.

Selecting Columns & Rows

```
# a single column (by name)
df["population"]

# several columns
df[["city", "population"]]

# rows by position with .iloc
df.iloc[0]      # the first row
df.iloc[0:3]    # the first three rows
```

A single column is called a **Series**. Selecting columns uses their **names**; selecting rows by position uses `.iloc`. (There's also `.loc` for selecting by label.)

Filtering Rows by Condition

The most common data task: keep only the rows that meet a condition. pandas uses the same boolean idea as NumPy.

```
# cities with more than 100,000 people
big = df[df["population"] > 100000]

# combine conditions with & (and), | (or)
df[(df["population"] > 50000) &
    (df["city"] != "Utrecht")]
```

Two gotchas: use `&` and `|` (not `and/or`) for pandas conditions, and wrap each condition in **parentheses**.

New Columns & Grouping

Two everyday operations: **creating columns** and **grouping** to summarize.

```
# create a new column from existing ones
df["pop_thousands"] = df["population"] / 1000

# group and summarize: mean population per region
df.groupby("region")["population"].mean()
```

groupby is the powerhouse of data analysis: split the data into groups, compute something per group (mean, sum, count...), and combine the results. It answers “what’s the average X for each Y?”

Quick Plots from a DataFrame

pandas has plotting built in — one method, and you get a chart (it uses matplotlib underneath).

```
# a bar chart of population by city
df.plot(x="city", y="population", kind="bar")

# a histogram of a numeric column
df["population"].plot(kind="hist")
```

kind can be "bar", "line", "hist", "scatter", and more. Great for a quick look; for polished figures you'd use matplotlib or seaborn directly.

Module 7: Takeaways

- ✓ `import pandas as pd`; the **DataFrame** is a programmable table
- ✓ `pd.read_csv()` loads data; `head()` `info()` `describe()` explore it
- ✓ Select columns by **name**, rows by position with `.iloc`
- ✓ **Filter** with boolean conditions (use `&` `|` and parentheses)
- ✓ Create columns, and `groupby()` to summarize per group
- ✓ `.plot()` gives quick charts straight from the DataFrame

That's the toolkit! You now know enough Python — variables, structures, control flow, functions, strings, NumPy, and pandas — to start real data analysis.